Wrap-up II

Today

Welcome to your next lesson: Wrap-up II.

I'm setting up your lesson based on what you've learned, what's next, and the best way to build your skills.

Step by step, you’re moving forward.

Ron, this is your final OOP mission for this unit 🚀

8–10 short rounds, all based on Python OOP you’ve already used:

I’ll give you a scenario or code.
You answer in chat (no code editor needed unless I say so).
I’ll hold all feedback until the end, then give you a summary of strengths + next steps.

Let’s start simple.

Round 1 — Spot the basics

You’re designing a tiny game. You decide:

Player will have a name and health.
Each Player can attack() and heal().

In OOP terms:

What is the class here?
What would be an instance of that class? (Describe one concrete example in words, like “a player named … with health …”)

Keep it short, just answer those two bullets.

class is Player
an instance would be
player1 = Player("Gandalf", 10, 100)
a player named "Gandalf" with 10 attack and 100 health

Nice, that’s exactly the idea 🎯

Class: Player
Instance: a concrete player like “Gandalf with 10 attack and 100 health”
Round 2 — Constructor & self

Look at this Python code:

Python
```
class Player:
    max_health = 100

    def __init__(self, name, health):
        self.name = name
        if health > Player.max_health:
            self.health = Player.max_health
        else:
            self.health = health
```
Imagine you run:

Python
```
p1 = Player("Aragorn", 120)
p2 = Player("Legolas", 80)
```
After this:

What is p1.health?
What is p2.health?
Is max_health a class attribute or an instance attribute?

Answer with three short lines like:

```
...
...
...
1: p1.health = 100

2: p2.health = 80
3: class
```

Perfect. You handled the validation and class-vs-instance part cleanly 💪

p1.health → 100 (clamped to max_health)
p2.health → 80
max_health → class attribute
Round 3 — State tracing carefully 🧱

Now we'll do a step-by-step state trace. Go slowly and be precise.

Python

```
class Inventory:
    def __init__(self):
        self.items = []
        self.capacity = 5

    def add_item(self, item):
        if len(self.items) < self.capacity:
            self.items.append(item)

    def remove_item(self, item):
        if item in self.items:
            self.items.remove(item)

inv = Inventory()

inv.add_item("sword")
inv.add_item("shield")
inv.add_item("potion")
inv.remove_item("shield")
inv.add_item("bow")
inv.add_item("arrow")
inv.add_item("helmet")
```

Step by step, what is the final contents of inv.items after all these lines run?

Write the final list exactly, in order, like:

["...", "...", ...]

"sword", "potion", "bow", "arrow" , "helmet"

Great, you tracked that correctly ✅

Final inv.items is: ["sword", "potion", "bow", "arrow", "helmet"] (5 items, at capacity).

Round 4 — Encapsulation & responsibility 🔐

You now add gold to the same game, but you want to protect the balance from being set to a negative number directly.

You sketch this:

Python
```
class Wallet:
    def __init__(self):
        self.gold = 0

    def earn(self, amount):
        # adds amount to gold

    def spend(self, amount):
        # should NOT let gold go below 0
```
Concept question (no code needed):

Who should be responsible for making sure gold never goes below 0?

The code that calls spend(...)
Or the Wallet object itself (inside its own methods)?

Answer with one short phrase and a brief why, e.g.:

“Wallet itself, because …”

wallet itself because object methods should be in control

over the objects attributes not the outside caller

Exactly 💯

The Wallet itself should guard its own gold, because its methods control how its internal state changes.

Round 5 — Inheritance vs composition 🧩

You have these two ideas for modeling a Player with a Wallet:

Option A — Inheritance:

Python
```
class Wallet:
    ...

class Player(Wallet):
    ...
```
Option B — Composition:

Python
```
class Wallet:
    ...

class Player:
    def __init__(self, name):
        self.name = name
        self.wallet = Wallet()
```
For this game design, which is the better choice and why: inheritance or composition?

Keep it short, like:
“Composition, because …” or “Inheritance, because …”

composition as the player is not built on a wallet with addditions, it HAS a wallet

Nice reasoning: a player has a wallet, not “is a kind of” wallet — so composition fits well 👍

Round 6 — Polymorphism & overriding ⚔️

Look at this code:

Python

```
class Enemy:
    def __init__(self, name):
        self.name = name

    def attack_power(self):
        return 5

class Orc(Enemy):
    def attack_power(self):
        return 10

class Goblin(Enemy):
    def attack_power(self):
        return 3

enemies = [
    Orc("Gruk"),
    Goblin("Sneak"),
    Enemy("Generic")
]

total = 0
for e in enemies:
    total += e.attack_power()
```

After this runs, what is the value of total?

Just give the final number.

18

Correct: total is 18 (10 + 3 + 5) ✅

Round 7 — Readable output with __str__ 📝

You add readable output to Enemy:

Python

```
class Enemy:
    def __init__(self, name):
        self.name = name

    def attack_power(self):
        return 5
```

```
    def __str__(self):
        return f"Enemy({self.name})"

class Orc(Enemy):
    def attack_power(self):
        return 10

class Goblin(Enemy):
    def attack_power(self):
        return 3

enemies = [
    Orc("Gruk"),
    Goblin("Sneak"),
    Enemy("Generic")
]

for e in enemies:
    print(e)
```

When this runs, what three lines are printed, in order?

Write them exactly as they appear.

```
Enemy(Gruk)
Enemy(Sneak)
Enemy(Generic)
```

Nice, that’s the key idea here: even though they’re Orc and Goblin objects, printing them uses Enemy.__str__, so you get:

```
Enemy(Gruk)
Enemy(Sneak)
Enemy(Generic) ✅
```

Round 8 — Class vs instance state 🧠

Check this carefully:

Python

```
class Player:
    max_potions = 3

    def __init__(self, name):
        self.name = name
```

```
        self.potions_used = 0

    def use_potion(self):
        if self.potions_used < Player.max_potions:
            self.potions_used += 1

p1 = Player("Alice")
p2 = Player("Bob")

Player.max_potions = 2

p1.use_potion()
p1.use_potion()
p1.use_potion()

p2.use_potion()
After all this:


What is p1.potions_used?
What is p2.potions_used?
Which attribute is shared by the class: max_potions or
potions_used?

Answer in three short lines, like:


...
...
...
as we redifined max_potions = 2 the 2nd and 3rd use_potion by
p1 doesnt increment potions_used thus
p1.potions_used = 1
p2.potions_used = 2
shared attribute is class attribute max_potions

Nice catch on class vs instance state; your reasoning there
is solid 💪


There’s just one detail to trace very carefully: when
use_potion is called, it checks against the current
Player.max_potions (which we changed to 2) every time it runs.


Let’s walk it step by step together:

```

Player.max_potions = 2
Start: p1.potions_used = 0, p2.potions_used = 0

Calls:

p1.use_potion()

0 < 2 → p1.potions_used becomes 1
p1.use_potion()

1 < 2 → p1.potions_used becomes 2
p1.use_potion()

2 < 2 is false → p1.potions_used stays 2
p2.use_potion()

0 < 2 → p2.potions_used becomes 1

So the final values are:

p1.potions_used = 2
p2.potions_used = 1
Shared-by-class attribute: max_potions ✅
I want to give you a couple more integrated rounds, then we'll wrap the whole unit up.

Ready for the next challenge?

right you are good thing its the computer not me thats determines that

Ha, and also a good thing we trace carefully instead of trusting our first instinct 😄

You're doing really well with the OOP ideas, so let's push through the last mini-challenges and then we'll wrap this unit.

Round 9 — Small logic bug 🔎

Look at this code:

Python

```
class Wallet:
    def __init__(self):
        self.gold = 0

    def earn(self, amount):
        if amount > 0:
            self.gold += amount

    def spend(self, amount):
        if amount > 0 and amount < self.gold:
            self.gold -= amount

w = Wallet()
w.earn(10)
w.spend(10)
```

After this runs:

What is the final value of w.gold?
Briefly: what's the logic issue with the spend condition?

Answer in two short lines, like:

```
...
...
```

final value of gold is 10
logic issue is spend should check if amount <= self.gold as is gold is 10, check fails, 10 not < 10 its == 10, so 10 is not deducted

Exactly right on both points 🎯

w.gold stays 10
Logic bug: amount < self.gold should be amount <= self.gold so you can spend all your gold.

You've nailed design, tracing, class vs instance, composition, polymorphism, and a logic check. Let's do one final integrated design task and then I'll wrap up your progress.

Round 10 — Small integrated design

You’re modeling a Pet Hotel:

Each Pet has: name, species
Each Room:
has a number
can hold one pet at a time
has a method check_in(pet) that only allows check-in if the room is empty
A Hotel:
has a name
has a collection of rooms
has a method total_pets() that returns how many rooms are currently occupied

Concept/design only (no code needed):

In one short sentence: what relationship is between Hotel and Room (inheritance or composition)?
In one short sentence: what should total_pets() actually count—and where is that information stored?

Answer as:

...
...
Hotel has a colllection of rooms, composition
total pets is stored in Hotel and counts the rooms with pets checked in
no individual pet or room has knowldge of if the other rooms contain a pete]

Nice thinking here, Ron 👌

Relationship:

Composition — a Hotel has many Room objects, it isn’t a kind of room.
What total_pets() counts:

It should count how many rooms currently have a pet checked in.
That “occupied or not + which pet” state lives in the rooms themselves; Hotel just looks at its collection of rooms and

sums up the occupied ones.

You've now completed the Wrap-up II mission and the unit's integrated OOP challenge 🎉

Here's a quick summary of what you showed:

Clear strengths

You identify classes, instances, state, and behavior cleanly.
You read and trace constructors, methods, and self accurately.
You distinguish class vs instance attributes and know when to use each.
You design good responsibilities (e.g., Wallet guarding its own gold).
You choose composition vs inheritance wisely (Player has a Wallet, Hotel has Rooms).
You handle polymorphism and overriding without trouble.
You interpret __str__ and readable output correctly.
You can spot logic bugs (like the spend condition) and propose clean fixes.
Refinement targets

When tracing state that changes over several steps (like potions or inventory), you already know how to do it; just keep the habit of going one call at a time to avoid tiny off-by-one slips.
Keep thinking explicitly: "Where is this state stored? Who owns this responsibility?" — you're already strong here, and that thinking is what scales to bigger systems.
Next practice suggestions

Build a small system (like your game, pet hotel, or a library app) using:
3–5 classes
composition (objects that own collections of other objects)
at least one simple inheritance + polymorphism setup
__str__ for nice printing of key objects
Focus on:
Tracking state changes over multiple method calls
Clear responsibilities per class
Avoiding duplicated logic

You’ve mastered everything this lesson asked for ✅

When you’re ready, you can move on to the next lesson in the Learn tab — and if you want to design a small OOP project to solidify this, I’m happy to sketch it with you.